Guide to Generating StationXML for DIY Seismometers

Introduction

This document provides a comprehensive guide to generating StationXML metadata for a DIY seismometer system, specifically one consisting of a geophone, an ADS1256 Analog-to-Digital Converter (ADC), and a Raspberry Pi. StationXML is a standardized XML format used in seismology to describe seismic stations, their sensors, and data loggers, along with their response characteristics. Accurate StationXML is crucial for proper data interpretation and exchange within the seismological community.

This guide will cover:

1. **Understanding StationXML**: A brief overview of the StationXML format and its importance.
2. **Component Specifications**: Key parameters and characteristics of the geophone, ADS1256, and Raspberry Pi relevant to StationXML.
3. **StationXML Template and Structure**: Explanation of the essential elements within a StationXML file.
4. **Python Tool for Generation**: How to use the provided Python script (`generate_stationxml.py`) to create your StationXML file.
5. **Interpreting Response Information**: Understanding poles, zeros, and gain values in the context of your DIY seismometer.

1. Understanding StationXML

StationXML is the current standard for exchanging station metadata within the FDSN (International Federation of Digital Seismograph Networks) community. It provides a detailed description of a seismic station, including its location, the instruments deployed (sensors and data loggers), and their complete instrumental response. This response information is critical for converting raw digital counts from the seismometer into ground motion units (e.g., meters or meters/second).

Without accurate StationXML, seismic data is essentially meaningless. Researchers and data users rely on this metadata to understand how the instrument recorded the ground

motion, allowing them to correct for instrumental effects and compare data from different stations and instruments.

2. Component Specifications

To accurately describe your DIY seismometer in StationXML, you need to understand the key specifications of each component:

Geophone

A geophone is a passive velocity sensor that converts ground velocity into an electrical voltage. Key parameters for StationXML include:

- **Model and Serial Number**: Unique identifiers for your specific geophone.
- **Natural Frequency (Fn)**: The resonant frequency of the geophone. For example, a 10 Hz geophone has a natural frequency of 10 Hz.
- **Damping (h)**: A dimensionless quantity describing how oscillations in the geophone decay after a disturbance. Often expressed as a damping ratio (e.g., 0.707 for critical damping).
- **Generator Constant (G)**: Also known as sensitivity, this is the conversion factor from ground velocity to output voltage (e.g., V/(m/s)). This is a crucial value for determining the overall instrument sensitivity.
- **Poles and Zeros**: These mathematical terms describe the frequency response of the geophone. They are typically provided in the geophone's datasheet. For a simple geophone, you'll usually have two poles (complex conjugate pair) and two zeros at the origin (0+0j).

ADS1256 (Analog-to-Digital Converter)

The ADS1256 is a high-precision, 24-bit ADC that converts the analog voltage from the geophone into digital counts that the Raspberry Pi can process. Key parameters for StationXML include:

- **Model and Serial Number**: Unique identifiers for your specific ADS1256 board.
- **Reference Voltage (Vref)**: The voltage range the ADC uses for conversion. For example, if your ADS1256 is configured for a 2.5V reference, then 2.5V corresponds to the full-scale digital output.
- **Gain (PGA setting)**: The programmable gain amplifier (PGA) on the ADS1256 amplifies the input signal. This gain setting directly affects the conversion factor from voltage to counts. For example, a PGA of 1 means no amplification, while a PGA of 2 amplifies the signal by a factor of 2.

- **Number of Bits**: The resolution of the ADC (e.g., 24-bit). This determines the total number of possible digital counts.

Raspberry Pi

The Raspberry Pi acts as the data logger, collecting the digital data from the ADS1256 and storing it. While less critical for the instrumental response itself, its role is important for the overall station description:

- **Model and Serial Number**: Identifiers for your Raspberry Pi.
- **Operating System and Software**: Details about the software used for data acquisition and logging.
- **Installation Date**: When the system was set up.

3. StationXML Template and Structure

A StationXML file is structured hierarchically, starting with an `Inventory` that contains one or more `Network` elements. Each `Network` can contain multiple `Station` elements, and each `Station` can have multiple `Channel` elements. The `Channel` element is where the detailed instrument response information is provided.

Here's a simplified breakdown of the key elements and how they relate to your DIY seismometer:

- **Inventory**: The root element, containing all metadata.
  - `source`: The organization or project creating the metadata (e.g., "DIY Seismometer Project").
- **Network**: Describes a group of stations.
  - `code`: A two-character network code (e.g., "XX" for experimental).
  - `description`: A human-readable description of the network.
  - `startDate`: The date the network became active.
- **Station**: Describes a single seismic station.
  - `code`: A five-character station code (e.g., "DIY01").
  - `latitude`, `longitude`, `elevation`: Geographic coordinates and elevation of the station.
  - `description`: A description of the station (e.g., "DIY Seismometer Station with Geophone, ADS1256, and Raspberry Pi").
  - `creationDate`: The date the station was established.
  - `Site`: Contains information about the physical location.
    - `Name`: A descriptive name for the site (e.g., "Home Lab").

- **Channel**: Describes a single data channel (e.g., vertical component).
  - `code`: A three-character channel code (e.g., "HHZ" for high-broadband, high-gain, vertical).
  - `locationCode`: A two-character location code (often empty for single sensors).
  - `latitude`, `longitude`, `elevation`, `depth`: Precise coordinates of the sensor within the station.
  - `azimuth`, `dip`: Orientation of the sensor.
  - `sampleRate`: The sampling rate of the digital data (e.g., 100.0 Hz).
  - `startDate`: The date the channel became active.
  - `Sensor`: Describes the physical sensor (geophone).
    - `Type`, `Description`, `Manufacturer`, `Model`, `SerialNumber`, `InstallationDate`.
  - `DataLogger`: Describes the digitizer (ADS1256) and data acquisition system (Raspberry Pi).
    - `Type`, `Description`, `Manufacturer`, `Model`, `SerialNumber`, `InstallationDate`.
  - `Response`: Contains the instrumental response information.
    - `InstrumentSensitivity`: The overall sensitivity of the instrument, typically in Counts per physical unit (e.g., Counts/(m/s)). This is the product of the geophone's sensitivity and the ADC's gain.
      - `Value`: The calculated sensitivity.
      - `Frequency`: The frequency at which the sensitivity is valid (often 1.0 Hz).
      - `InputUnits`: Units of the physical input (e.g., "M/S" for velocity).
      - `OutputUnits`: Units of the digital output (e.g., "COUNTS").
    - `PolesZerosResponseStage`: Describes the frequency response of the geophone using poles and zeros.
      - `stageSequenceNumber`: Order of this stage in the overall response chain.
      - `stageGain`: Gain of this stage.
      - `pzTransferFunctionType`: Type of transfer function (e.g., "LAPLACE (HERTZ)").
      - `inputUnits`, `outputUnits`: Units before and after this stage.
      - `Poles`, `Zeros`: Lists of complex numbers representing the poles and zeros.
      - `NormalizationFactor`, `NormalizationFrequency`: Used for scaling the response.

- `CoefficientsTypeResponseStage`: Describes the gain of the ADC.
  - `stageSequenceNumber`: Order of this stage.
  - `stageGain`: Gain of this stage (e.g., the ADC's gain in Counts/Volt).
  - `inputUnits`, `outputUnits`: Units before and after this stage.
  - `cfTransferFunctionType`: Type of transfer function (e.g., "DIGITAL").
  - `Numerator`, `Denominator`: Coefficients of the transfer function (for a simple gain, numerator is the gain, denominator is 1.0).

4. Python Tool for Generation (`generate_stationxml.py`)

We have developed a Python script, `generate_stationxml.py`, that utilizes the ObsPy library to create a StationXML file for your DIY seismometer. ObsPy is a powerful open-source framework for seismological data analysis, and it includes robust tools for handling StationXML.

Prerequisites

Before running the script, ensure you have ObsPy installed:

```
pip3 install obspy
```

Script Usage

The `generate_stationxml.py` script defines a function `create_stationxml` that takes various parameters related to your station, channel, and instrument response. The `if __name__ == "__main__":` block at the end of the script provides an example of how to call this function with placeholder values.

Key parameters to customize in the `if __name__ == "__main__":` block:

- `network_code`, `station_code`, `station_latitude`, `station_longitude`, `station_elevation`, `station_start_date`
- `channel_code`, `channel_latitude`, `channel_longitude`, `channel_elevation`, `channel_depth`, `channel_azimuth`, `channel_dip`, `sample_rate`
- `geophone_model`, `geophone_serial`, `geophone_installation_date`

- `ads1256_serial`, `ads1256_installation_date`
- `instrument_sensitivity_value`, `instrument_sensitivity_frequency`
- `geophone_poles`, `geophone_zeros`, `geophone_gain`
- `ads1256_gain`

Example Calculation for `instrument_sensitivity_value` and `ads1256_gain`:

These values are crucial and require careful calculation based on your specific components. Here's a breakdown of the example calculations in the script:

- **`instrument_sensitivity_value` (Overall System Sensitivity - Counts per m/s):** This represents the total conversion from ground velocity to digital counts. It's typically the product of your geophone's sensitivity (V/(m/s)) and your ADC's gain (Counts/V).
    - **Geophone Sensitivity**: This is a value you'll find in your geophone's datasheet (e.g., 75.7 V/m/s for a hypothetical geophone).
    - **ADS1256 Counts per Volt**: This depends on the ADS1256's reference voltage and its resolution. For a 24-bit ADC with a 2.5V bipolar reference, the calculation is: `Counts per Volt = (2^23) / 2.5 = 3355443.2 Counts/Volt` (since 24-bit bipolar means 2^23 counts for positive and 2^23 for negative, so 2^23 for 2.5V range).
    - **`instrument_sensitivity_value` = Geophone Sensitivity (V/m/s) * ADS1256 Counts per Volt (Counts/V)** `= 75.7 V/m/s * 3355443.2 Counts/V = 253987800.0 Counts/m/s`
- **`ads1256_gain` (ADC Gain - Counts/Volt):** This is the gain of your ADS1256, converting voltage to counts. If your PGA is set to 1, this is simply the `Counts per Volt` calculated above. If your PGA is set to a different value (e.g., 2, 4, 8, etc.), you would multiply the `Counts per Volt` by the PGA setting.

  `ads1256_gain = 3355443.2 # Counts/Volt (for 2.5V Vref, 24-bit bipolar)`

Running the script:

After customizing the parameters, execute the script from your terminal:

```
python3 generate_stationxml.py
```

This will generate a file named `diy_seismometer_station.xml` in the same directory.

5. Interpreting Response Information

The `Response` section in StationXML is the most complex but also the most important. It describes how the instrument modifies the ground motion signal as it passes through the sensor and data logger. This is typically represented as a series of

stages, each with its own transfer function. For your DIY seismometer, the primary stages are:

Geophone Response (Poles and Zeros)

The geophone's response is typically described by its poles and zeros. These are complex numbers that define the frequency-dependent behavior of the sensor. The `PolesZerosResponseStage` in StationXML captures this information.

- **Poles**: Represent frequencies at which the system's response tends towards infinity. For a geophone, these are related to its natural frequency and damping.
- **Zeros**: Represent frequencies at which the system's response tends towards zero. For a velocity geophone, there are typically two zeros at the origin (0+0j), indicating that it does not respond to constant displacement (zero frequency).
- **Normalization Factor and Frequency**: These values ensure that the response is correctly scaled. The normalization factor is often the geophone's gain (V/m/s) at a specific normalization frequency (e.g., 1 Hz).

Finding your Geophone's Poles and Zeros:

This is the most challenging part for a DIY seismometer. Ideally, these values are provided in the geophone's datasheet. If you are using a generic geophone without a datasheet, you might need to:

1. **Search for similar geophones**: Look for datasheets of commercially available geophones with similar specifications (e.g., natural frequency, coil resistance). This can give you a starting point for typical pole and zero locations.
2. **Experimental Determination**: This is more advanced and involves specialized equipment to characterize the geophone's frequency response. This is beyond the scope of this guide but is the most accurate method.

For the example in `generate_stationxml.py`, we used simplified poles and zeros for a hypothetical 10 Hz geophone. **It is crucial to replace these with values specific to your geophone for accurate metadata.**

ADS1256 Response (Coefficients)

The ADS1256 acts as a digitizer, converting the analog voltage from the geophone into digital counts. Its response is typically a simple gain, which can be represented by a `CoefficientsTypeResponseStage`.

- **`stageGain`**: This is the gain of the ADS1256, converting Volts to Counts. As calculated in the example, this depends on the reference voltage and the PGA setting.
- **`cf_transfer_function_type`**: For an ADC, this is typically "DIGITAL".
- **`numerator` and `denominator`**: For a simple gain, the numerator will be a list containing your `ads1256_gain` value, and the denominator will be `[1.0]`.

Overall Instrument Sensitivity

The `InstrumentSensitivity` in StationXML represents the total sensitivity of the entire instrument chain (geophone + ADS1256). It's the product of the sensitivities of all stages. In our case, it's the geophone's sensitivity (V/m/s) multiplied by the ADS1256's gain (Counts/V), resulting in a sensitivity in Counts/(m/s).

Conclusion

Generating accurate StationXML metadata is a vital step in building a functional DIY seismometer. While some parameters, like geophone poles and zeros, might require careful research or experimental determination, the provided Python script and this guide offer a solid foundation. By meticulously filling in the correct specifications for your components, you will create a StationXML file that enables proper interpretation and use of your seismic data.

References

[1] FDSN StationXML Schema: https://www.fdsn.org/xml/station/ [2] ObsPy Tutorial: Creating a StationXML file from Scratch: https://docs.obspy.org/tutorial/code_snippets/stationxml_file_from_scratch.html [3] ObsPy Documentation: `obspy.core.inventory` module: https://docs.obspy.org/packages/obspy.core.inventory.html